

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN
PHÁT TRIỂN ỨNG DỤNG IOT

GVHD:

SVTH: Thạch Minh Hưng – 2311357
Nguyễn Lê Khôi Nguyên – 2312366

BẢNG PHÂN CÔNG CÔNG VIỆC CỦA NHÓM

Tên thành viên	MSSV	Công việc	Mức độ
Thạch Minh Hưng	2311357	- Lập trình Module: LCD, Buzzer. - Thiết kế, hiện thực giao diện LCD. - Viết báo cáo.	100%
Nguyễn Lê Khôi Nguyên	2312366	- Hiện thực chức năng thông báo. - Lập trình Module: LED 7 đoạn. - Viết báo cáo tổng hợp.	100%

LỜI CẢM ƠN

Lời đầu tiên, nhóm xin gửi lời cảm ơn chân thành đến **Khoa Khoa học và Kỹ thuật Máy** đã tạo điều kiện thuận lợi và cung cấp môi trường học tập, nghiên cứu tốt trong suốt thời gian qua.

Nhóm xin bày tỏ lòng biết ơn sâu sắc đến Thầy **Lê Trọng Nhân** — giảng viên hướng dẫn **Bài Tập Lớn môn Phát Triển Ứng Dụng IOT (CO3037)**. Trong quá trình thực hiện đề tài, nhóm đã nhận được sự hướng dẫn tận tình, những góp ý kịp thời và định hướng rõ ràng từ Thầy. Các kiến thức về hệ thống nhúng, vi điều khiển và phương pháp tư duy thiết kế mà Thầy truyền đạt là nền tảng quan trọng giúp nhóm từng bước hoàn thiện hệ thống và triển khai trên Kit thí nghiệm **Yolo Uno**.

Thông qua đồ án Bài Tập Lớn này, nhóm không chỉ củng cố kiến thức lý thuyết mà còn rèn luyện kỹ năng làm việc nhóm, kỹ năng phân tích – giải quyết vấn đề, cũng như tiếp cận và ứng dụng các công nghệ mới.

Dù đã cố gắng hoàn thành báo cáo một cách nghiêm túc, nhưng do giới hạn về thời gian, tài nguyên và kinh nghiệm, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý từ Thầy để đề tài được hoàn thiện hơn và giúp nhóm rút ra thêm kinh nghiệm cho các dự án sau này.

Cuối cùng, nhóm xin kính chúc Thầy dồi dào sức khỏe và nhiều thành công trong công tác giảng dạy, nghiên cứu.

Nhóm xin chân thành cảm ơn!

Mục lục

LỜI CẢM ƠN	ii
1 GIỚI THIỆU VÀ MỤC TIÊU	1
1.1 Giới thiệu đề tài	1
1.2 Mục tiêu đề tài	1
2 MÔ TẢ CHỨC NĂNG CÁC TÁC VỤ	2
2.1 Task 1: LED Nhấp Nháy Đơn Với Điều Khiển Nhiệt Độ	2
2.2 Task 2: Điều Khiển LED NeoPixel Dựa Trên Độ Ẩm	2
2.2.1 Cảm biến DHT20 so với các cảm biến khác	2
2.2.2 Triển khai và Code	3
2.3 Task 3: Giám Sát Nhiệt Độ Và Độ Ẩm Với Màn Hình LCD	5
2.4 Task 4: Web Server Ở Chế Độ Access Point	5
2.4.1 Wifi	5
2.4.2 Webserver	11
2.4.3 Triển khai và Code	11
2.5 Task 5: Triển Khai TinyML & Đánh Giá Độ Chính Xác	11
2.6 Task 6: Xuất Bản Dữ Liệu Lên Máy Chủ Cloud CoreIOT	11
3 ĐIỂM NHẤN TRIỂN KHAI	12
3.1 Logic Semaphore	12
3.2 Thiết Kế Lại Web Server	12
3.3 Quy Trình TinyML	12
3.4 Quy Trình CoreIOT	12
3.5 Các Công Nghệ Sử Dụng	12
3.6 Sơ Đồ Kiến Trúc Hệ Thống	12
4 PHẦN CỨNG VÀ CÔNG CỤ SỬ DỤNG	13
4.1 Danh sách các thành phần phần cứng	13
4.2 Công cụ phát triển	13
4.3 Môi trường phát triển	13
5 ĐÁNH GIÁ VÀ KẾT QUẢ THỬ NGHIỆM	14
5.1 Kết Quả Kiểm Thử Các Tính Năng	14

5.2	Đánh Giá Độ Chính Xác TinyML	14
5.3	Hiệu Năng Hệ Thống	14
5.4	Các Khó Khăn và Giải Pháp Khắc Phục	14
6	THẢO LUẬN NHÓM VÀ KẾT LUẬN	15
6.1	Kết Quả Dự Án Đạt Được	15
6.2	Thảo Luận Nhóm	15
6.3	Hạn Chế Của Dự Án	15
6.4	Hướng Phát Triển Trong Tương Lai	15
7	THÔNG TIN NHÓM	16
7.1	Tên Và Vai Trò Các Thành Viên Nhóm	16
7.2	Liên Kết Repository GitHub	16
7.3	Thông Tin Liên Hệ	16

1 GIỚI THIỆU VÀ MỤC TIÊU

1.1 Giới thiệu đề tài

Trong bối cảnh phát triển mạnh mẽ của Internet vạn vật (IoT) và trí tuệ nhân tạo, đề tài tập trung xây dựng hệ thống nhà thông minh để nhận diện và điều khiển tự động các thiết bị ngoại vi bao gồm đèn, quạt và cửa. Bên cạnh khả năng tương tác thông minh dựa trên thị giác máy tính, hệ thống còn tích hợp các module cảm biến để giám sát thời gian thực các thông số môi trường như nhiệt độ và độ ẩm. Sự kết hợp này không chỉ hướng tới việc tối ưu hóa trải nghiệm tiện nghi và tính an toàn cho không gian sống hiện đại mà còn nâng cao hiệu quả quản lý năng lượng một cách chủ động và thông minh.

1.2 Mục tiêu đề tài

Mục tiêu chính của đề tài là thiết kế và xây dựng một hệ thống điều khiển nhà thông minh có khả năng tương tác linh hoạt giữa con người, môi trường và thiết bị. Cụ thể, đề tài hướng tới các mục tiêu chi tiết sau:

- **Về mặt thiết bị:** Triển khai dự án nhà thông minh sử dụng kit thí nghiệm Yolo Uno và các thiết bị ngoại vi như đèn, quạt và cửa, đồng thời tích hợp các module cảm biến để thu thập dữ liệu môi trường.
- **Về mặt giám sát:** Tích hợp các module cảm biến để thu thập và hiển thị chính xác các thông số môi trường bao gồm nhiệt độ và độ ẩm trong thời gian thực.
- **Về mặt hệ thống:** Xây dựng mô hình hoạt động ổn định, đảm bảo độ trễ thấp trong quá trình xử lý tính hiệu và phản hồi của các thiết bị ngoại vi.
- **Về mặt ứng dụng:** Tối ưu hóa trải nghiệm người dùng thông qua giao diện thân thiện và dễ sử dụng nhất.

2 MÔ TẢ CHỨC NĂNG CÁC TÁC VỤ

2.1 Task 1: LED Nhập Nháy Đơn Với Điều Kiện Nhiệt Độ

2.2 Task 2: Điều Khiển LED NeoPixel Dựa Trên Độ Ẩm

2.2.1 Cảm biến DHT20 so với các cảm biến khác

Trong phần này, cảm biến DHT20 được phân tích dựa trên kết quả thực nghiệm trong buồng khí hậu với dải nhiệt độ từ 10 đến $+40^{\circ}\text{C}$ và độ ẩm từ 0 đến 90%. Kết quả cho thấy DHT20 là một lựa chọn tối ưu cho các hệ thống giám sát môi trường nhờ sự kết hợp giữa độ chính xác và tiết kiệm năng lượng.

Độ chính xác đo lường Nhiệt độ và Độ ẩm Dựa trên các chỉ số thống kê như hệ số xác định (R^2) và sai số bình phương trung bình ($RMSE$), hiệu suất của DHT20 được so sánh cụ thể như sau:

Về đo lường Nhiệt độ (T_a): DHT20 đạt kết quả tốt nhất trong số tất cả các cảm biến giá rẻ được thử nghiệm (bao gồm DHT22, SHTC3, BME680, SHT85, SCD30 và DHT11). Cảm biến này có độ tuyến tính lý tưởng với $R^2 = 0.9999$. Sai số $RMSE$ của DHT20 đạt mức thấp nhất là 0.1836, vượt qua cả DHT22 ($RMSE = 0.2127$) và vượt trội hoàn toàn so với DHT11 ($RMSE = 1.8981$).

Về đo lường Độ ẩm tương đối (RH): DHT20 cho kết quả tương quan tốt với $R^2 = 0.9951$ và $RMSE = 2.7351$. Mặc dù DHT22 có hiệu suất đo độ ẩm nhanh hơn ($RMSE = 1.7396$), DHT20 vẫn được đánh giá cao về tính ổn định trong suốt 10 chu kỳ thử nghiệm. Một điểm lưu ý là DHT20 có thể gặp khó khăn nhất định khi bám sát các giá trị tham chiếu ở mức độ ẩm cực thấp hoặc cực cao.

Mức tiêu thụ năng lượng Tiêu thụ điện năng là một yếu tố then chốt cho các thiết bị IoT hoạt động bằng pin. DHT20 thể hiện ưu thế rõ rệt so với các thế hệ cũ:

- **Công suất tiêu thụ:** DHT20 có mức tiêu thụ điện năng tối đa chỉ 4.9 mW.
- **Dòng điện hoạt động:** Dòng điện cung cấp tối đa của DHT20 là 0.98 mA (tại điện áp 5 V).
- **So sánh:** Chỉ số này thấp hơn đáng kể so với DHT11 (12.5 mW) và DHT22 (7.5 mW), giúp kéo dài thời gian vận hành của hệ thống.

Bảng tổng hợp các thông số thực nghiệm Dưới đây là bảng so sánh các chỉ số đo lường chính giữa DHT20 và hai dòng cảm biến phổ biến cùng họ:

Kết luận Với đặc tính kỹ thuật đáp ứng yêu cầu của tiêu chuẩn ISO 7726 về đo lường nhiệt độ, DHT20 là sự thay thế hoàn hảo cho DHT11 trong các ứng dụng đòi hỏi độ tin cậy cao. Việc sử dụng phương trình hồi quy tuyến tính đơn giản có thể giúp hiệu chỉnh dữ liệu từ DHT20 tiệm cận sát với các thiết bị tham chiếu chuyên dụng, nâng cao độ chính xác cho toàn hệ thống.

Chỉ số đánh giá	DHT11	DHT22	DHT20
Sai số Nhiệt độ ($RMSE$)	1.8981	0.2127	0.1836 (Tốt nhất)
Hệ số xác định Nhiệt độ (R^2)	0.9839	0.9999	0.9999
Sai số Độ ẩm ($RMSE$)	17.7272	1.7396 (Tốt nhất)	2.7351
Công suất tối đa (mW)	12.5	7.5	4.9
Giao thức kết nối	Single-wire	Single-wire	I2C

Bảng 1: So sánh các chỉ số đo lường giữa DHT20 và các cảm biến khác

2.2.2 Triển khai và Code

Hệ thống được lập trình để ánh xạ giá trị độ ẩm tương đối (RH) đọc từ cảm biến DHT20 sang màu sắc của đèn NeoLED theo ba dải trạng thái chính. Cách thiết kế này giúp người dùng quan sát nhanh mức độ ẩm môi trường mà không cần đọc trực tiếp giá trị số trên màn hình.

Dải độ ẩm (RH)	Màu sắc NeoLED	Ý nghĩa trạng thái
< 30%	Đỏ (Red)	Độ ẩm thấp, khô hạn
30% – 60%	Vàng (Yellow)	Độ ẩm trung bình
> 60%	Xanh lá (Green)	Độ ẩm cao, ẩm ướt

Bảng 2: Quy tắc ánh xạ độ ẩm tương đối sang màu NeoLED

Lý do thiết lập các ngưỡng độ ẩm

- **< 30% là màu đỏ:** Độ ẩm dưới 30% được xem là rất khô, có thể gây kích ứng da, khô mắt và ảnh hưởng đến đường hô hấp. Màu đỏ đóng vai trò cảnh báo nguy hiểm, nhắc người dùng cần bổ sung ẩm kịp thời.
- **30% – 60% là màu vàng:** Đây là dải chuyển tiếp. Mặc dù dải lý tưởng thường nằm trong khoảng 40% – 60%, ngưỡng 30% – 60% được đặt màu vàng để biểu thị trạng thái cần chú ý hoặc bình thường.
- **> 60% là màu xanh lá:** Trong các ứng dụng như nhà kính, trồng cây nhiệt đới hoặc bảo quản vật liệu cần độ ẩm cao, mức trên 60% được xem là trạng thái tốt và đủ nước. Nếu áp dụng cho sinh hoạt gia đình, ngưỡng này có thể cần hiệu chỉnh tùy mục đích sử dụng.

Ưu điểm kỹ thuật của hệ thống

- **Độ tin cậy cao:** DHT20 đáp ứng các yêu cầu đo lường môi trường theo tiêu chuẩn ISO 7726.
- **Tiết kiệm năng lượng:** Cảm biến DHT20 có công suất tiêu thụ tối đa chỉ 4.9 mW (0.98 mA tại 5 V), phù hợp với thiết bị IoT chạy pin.

- **Tính tuyến tính tốt:** So với các cảm biến giá rẻ thế hệ cũ, DHT20 cho quan hệ đo lường ổn định hơn, giúp màu sắc hiển thị trên NeoLED phản ánh rõ hơn biến thiên độ ẩm.

Mã nguồn triển khai Đoạn mã dưới đây được tách thành hai file riêng để dễ bảo trì và đọc hơn. File header khai báo giao diện task, còn file source chứa logic điều khiển NeoLED theo giá trị *RH* đọc từ DHT20.

Listing 1: Header của task NeoLED

File header

```

1 #ifndef TASK_NEO_LED_H
2 #define TASK_NEO_LED_H
3
4 #include <Arduino.h>
5 #include <Adafruit_NeoPixel.h>
6 #include "global.h"
7
8 #define NEOPIXEL_PIN 45
9 #define LED_COUNT 1
10
11 void TaskNEOLED(void *pvParameters);
12
13 #endif // TASK_NEO_LED_H

```

Listing 2: Triển khai task NeoLED điều khiển màu đèn theo độ ẩm

File source

```

1 #include "task_neo_led.h"
2
3
4 void TaskNEOLED(void *pvParameters) {
5     Adafruit_NeoPixel strip(LED_COUNT, NEOPIXEL_PIN, NEO_GRB +
6         NEO_KHZ800);
7     strip.begin();
8     strip.clear();
9     strip.show(); // Initialize all pixels to 'off'
10
11     while(1) {
12         xSemaphoreTake(xBinarySemaphoreTemp_Humi, portMAX_DELAY); //
13             Wait for temperature and humidity data to be available
14         if(glob_humidity < 30){
15             strip.setPixelColor(0, strip.Color(100, 0, 0)); // Red
16             strip.show();
17         } else if(glob_humidity >= 30 && glob_humidity < 60){
18             strip.setPixelColor(0, strip.Color(100, 100, 0)); //
19                 Yellow
20             strip.show();
21         } else {
22             strip.setPixelColor(0, strip.Color(0, 100, 0)); // Green
23             strip.show();
24         }
25     }
26 }

```

Mã nguồn trên thể hiện đúng logic ánh xạ đã mô tả: $RH < 30\%$ thì NeoLED sáng đỏ, $30\% - 60\%$ thì vàng, và trên 60% thì xanh lá. Việc tách thành hai file riêng giúp cấu trúc dự án rõ ràng hơn và thuận tiện khi mở rộng thêm các trạng thái hiển thị khác.

2.3 Task 3: Giám Sát Nhiệt Độ Và Độ Ẩm Với Màn Hình LCD

2.4 Task 4: Web Server Ở Chế Độ Access Point

2.4.1 Wifi

ESP32 hỗ trợ nhiều chế độ Wi-Fi linh hoạt, cho phép thiết bị hoạt động như một trạm (Station), một điểm truy cập (Access Point), hoặc đồng thời cả hai. Việc chọn đúng chế độ giúp hệ thống phù hợp với từng tình huống triển khai IoT.

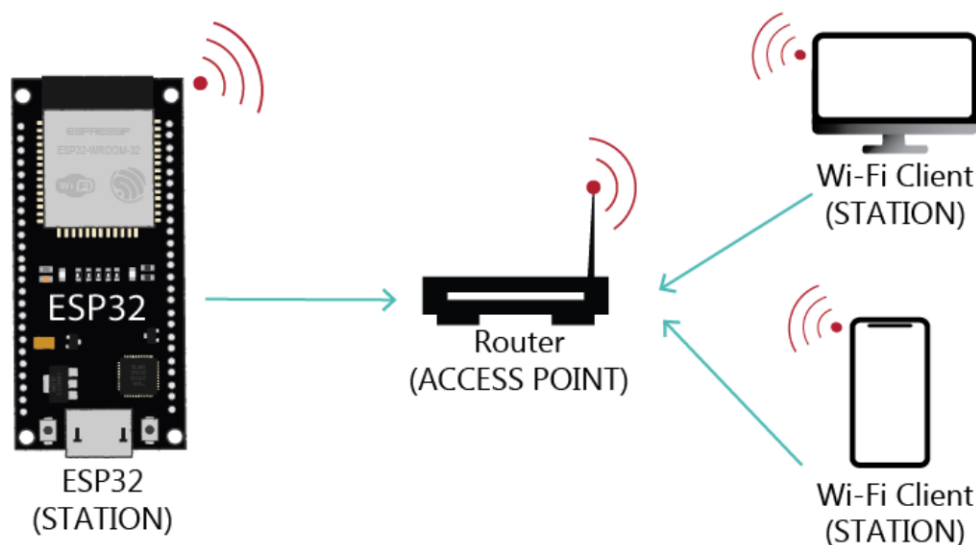
Các chế độ Wi-Fi của ESP32 Để thiết lập chế độ hoạt động Wi-Fi, sử dụng hàm:

```
1 WiFi.mode(...)
```

Các chế độ phổ biến:

- `WiFi.mode(WIFI_STA)`: Chế độ trạm (Station), ESP32 kết nối tới điểm truy cập/router.
- `WiFi.mode(WIFI_AP)`: Chế độ điểm truy cập (Access Point), các thiết bị khác kết nối trực tiếp tới ESP32.
- `WiFi.mode(WIFI_AP_STA)`: Hoạt động đồng thời ở cả hai chế độ AP và STA.

Chế độ trạm (Wi-Fi Station) Khi cấu hình ở chế độ `WIFI_STA`, ESP32 sẽ kết nối vào mạng Wi-Fi sẵn có (thường là router). Router sẽ cấp địa chỉ IP riêng cho ESP32, từ đó các thiết bị trong cùng mạng có thể giao tiếp với ESP32 thông qua địa chỉ IP này.



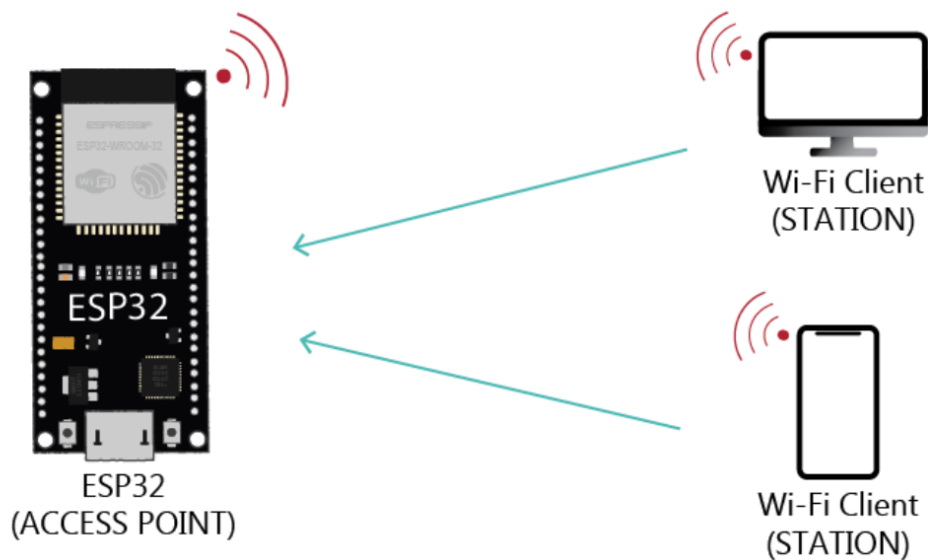
Hình 1: Mô hình kết nối Wi-Fi Station

Ưu điểm của chế độ này là ESP32 có thể truy cập Internet để:

- Gửi hoặc nhận dữ liệu từ API (ví dụ dữ liệu thời tiết).
- Đẩy dữ liệu cảm biến lên nền tảng đám mây.
- Sử dụng tài nguyên trực tuyến như thư viện JavaScript, biểu tượng, hình ảnh.

Tuy nhiên, nếu không có mạng Wi-Fi sẵn có gần đó, chế độ Station sẽ không phù hợp để điều khiển thiết bị cục bộ.

Chế độ điểm truy cập (Access Point) Khi thiết lập `WIFI_AP`, ESP32 tự tạo ra một mạng Wi-Fi riêng. Điện thoại, laptop hoặc các thiết bị ESP32 khác có thể kết nối trực tiếp vào mạng này mà không cần router.



Hình 2: Mô hình kết nối Wi-Fi Access Point

Đây là dạng *soft-AP* (điểm truy cập mềm), do đó phù hợp cho giao tiếp cục bộ. Tuy nhiên, vì không đi qua router nên thiết bị kết nối vào soft-AP thường không có Internet, dẫn đến:

- Không thể truy cập dịch vụ Internet từ các thiết bị đang nối vào AP của ESP32.
- Không phù hợp cho các tác vụ cần HTTP request ra ngoài hoặc đồng bộ cloud trực tiếp qua AP này.

Thiết lập cơ bản:

```
1 WiFi.mode(WIFI_AP);
2 WiFi.softAP(ssid, password);
```

Trong đó:

- **ssid**: tên mạng Wi-Fi do ESP32 phát ra (tối đa 63 ký tự).
- **password**: mật khẩu (tối thiểu 8 ký tự), có thể để NULL nếu muốn mạng mở.

Hàm `softAP()` cũng hỗ trợ tham số mở rộng:

```
1 WiFi.softAP(const char* ssid, const char* password,
2             int channel, int ssid_hidden, int
               max_connection)
```

Ý nghĩa tham số:

- `channel`: kênh Wi-Fi (1–13).
- `ssid_hidden`: 0 hiển thị SSID, 1 ẩn SSID.
- `max_connection`: số thiết bị kết nối đồng thời (1–4).

Chế độ kết hợp AP + STA ESP32 có thể vừa kết nối đến router, vừa phát Wi-Fi riêng:

```
1 WiFi.mode(WIFI_AP_STA);
```

Chế độ này phù hợp cho các ứng dụng cần cấu hình cục bộ qua AP nhưng vẫn duy trì kết nối Internet qua STA.

Quét mạng Wi-Fi lân cận ESP32 có thể quét các mạng trong vùng phủ sóng để kiểm tra tín hiệu hoặc xác nhận router có nằm trong phạm vi hoạt động hay không.

Hàm quét:

```
1 int n = WiFi.scanNetworks();
```

Sau khi quét, có thể truy xuất thông tin từng mạng:

- `WiFi.SSID(i)`: tên mạng (SSID).
- `WiFi.RSSI(i)`: cường độ tín hiệu thu được (RSSI).
- `WiFi.encryptionType(i)`: kiểu mã hóa mạng.

Các kiểu mã hóa thường gặp:

- `WIFI_AUTH_OPEN`
- `WIFI_AUTH_WEP`
- `WIFI_AUTH_WPA_PSK`
- `WIFI_AUTH_WPA2_PSK`
- `WIFI_AUTH_WPA_WPA2_PSK`
- `WIFI_AUTH_WPA2_ENTERPRISE`

Thông tin RSSI rất hữu ích khi đánh giá nguyên nhân mất kết nối do tín hiệu yếu, từ đó tối ưu vị trí đặt thiết bị hoặc cấu hình anten.

Kết nối ESP32 với mạng Wi-Fi cụ thể Để ESP32 kết nối vào một mạng Wi-Fi cụ thể, cần biết trước SSID và password của mạng đó. Ngoài ra, mạng phải nằm trong vùng phủ sóng của ESP32 (có thể kiểm tra trước bằng chức năng quét mạng).

Khai báo thông tin mạng:

```
1 // Replace with your network credentials
2 const char* ssid = "REPLACE_WITH_YOUR_SSID";
3 const char* password = "REPLACE_WITH_YOUR_PASSWORD";
```

Hàm khởi tạo kết nối Wi-Fi:

```
1 void initWiFi() {
2     WiFi.mode(WIFI_STA);
3     WiFi.begin(ssid, password);
4     Serial.print("Connecting to WiFi..");
5     while (WiFi.status() != WL_CONNECTED) {
6         Serial.print('.');
7         delay(1000);
8     }
9     Serial.println(WiFi.localIP());
10 }
```

Sau đó gọi hàm `initWiFi()` trong `setup()` để thực hiện kết nối khi thiết bị khởi động.

Nguyên lý hoạt động của hàm kết nối

- `WiFi.mode(WIFI_STA)`: đưa ESP32 về chế độ trạm để kết nối vào Access Point/router.
- `WiFi.begin(ssid, password)`: bắt đầu quá trình xác thực và kết nối tới mạng đã chỉ định.
- `while (WiFi.status() != WL_CONNECTED)`: lặp chờ cho đến khi kết nối thành công.
- `WiFi.localIP()`: lấy địa chỉ IP do router cấp sau khi kết nối thành công.

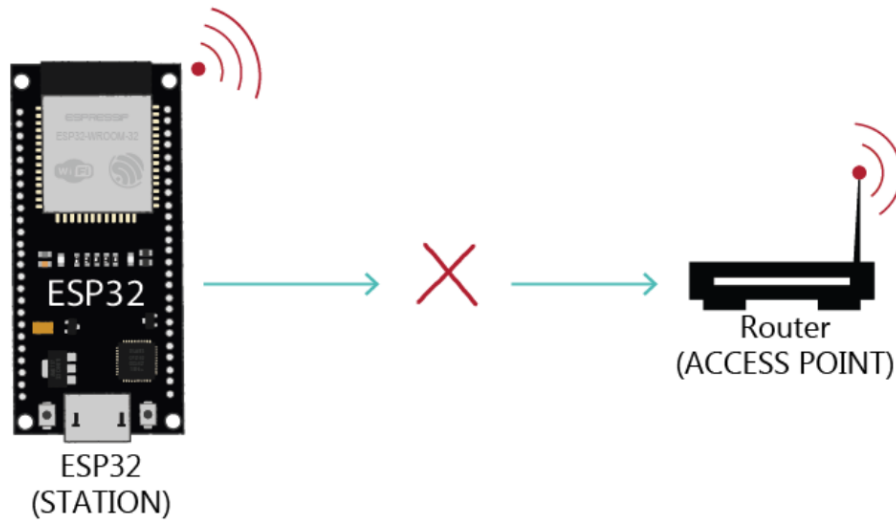
Kiểm tra trạng thái kết nối Wi-Fi Hàm `WiFi.status()` trả về trạng thái kết nối hiện tại. Bảng dưới đây mô tả các giá trị thường dùng:

Bảng 3: Các trạng thái kết nối Wi-Fi của ESP32

Giá trị	Hằng số	Ý nghĩa
0	WL_IDLE_STATUS	Trạng thái tạm thời khi vừa gọi <code>WiFi.begin()</code> .
1	WL_NO_SSID_AVAIL	Không tìm thấy SSID phù hợp.
2	WL_SCAN_COMPLETED	Quá trình quét mạng đã hoàn tất.
3	WL_CONNECTED	Kết nối Wi-Fi thành công.
4	WL_CONNECT_FAILED	Kết nối thất bại sau các lần thử.
5	WL_CONNECTION_LOST	Kết nối đã bị mất.
6	WL_DISCONNECTED	Thiết bị đang không kết nối mạng Wi-Fi.

Ngắt kết nối và kết nối lại Wi-Fi Để ngắt mạng đã kết nối trước đó, sử dụng:

```
1 WiFi.disconnect();
```



Hình 3: ESP32 đã ngắt kết nối Wi-Fi

Để thử kết nối lại tới AP đã dùng trước đó:

```
1 WiFi.reconnect();
```

Hoặc ngắt rồi kết nối lại bằng SSID/password:

```
1 WiFi.disconnect();
2 WiFi.begin(ssid, password);
```

Trong các hệ thống IoT chạy dài hạn, nên kiểm tra định kỳ và tự động reconnect:

```
1 unsigned long previousMillis = 0;
2 unsigned long interval = 30000;
3
4 void loop() {
5     unsigned long currentMillis = millis();
6     // if WiFi is down, try reconnecting
7     if ((WiFi.status() != WL_CONNECTED) && (currentMillis -
8         previousMillis >= interval)) {
9         Serial.print(millis());
10        Serial.println(" Reconnecting to WiFi...");
11        WiFi.disconnect();
12        WiFi.reconnect();
13        previousMillis = currentMillis;
14    }
15 }
```

Sự kiện Wi-Fi trên ESP32 ESP32 có thể phát sinh nhiều sự kiện liên quan đến Wi-Fi và Ethernet. Bảng sau tổng hợp các sự kiện thường gặp:

Bảng 4: Danh sách sự kiện Wi-Fi/Ethernet của ESP32

ID	Hàng số sự kiện	Ý nghĩa
0	ARDUINO_EVENT_WIFI_READY	ESP32 sẵn sàng kết nối Wi-Fi.
1	ARDUINO_EVENT_WIFI_SCAN_DONE	Hoàn tất quét các điểm truy cập (AP).
2	ARDUINO_EVENT_WIFI_STA_START	Khởi động trạm (STA).
3	ARDUINO_EVENT_WIFI_STA_STOP	Dừng trạm (STA).
4	ARDUINO_EVENT_WIFI_STA_CONNECTED	Trạm ESP32 đã kết nối AP.
5	ARDUINO_EVENT_WIFI_STA_DISCONNECTED	Trạm ESP32 bị ngắt khỏi AP.
6	ARDUINO_EVENT_WIFI_STA_AUTHMODE_CHANGE	Chế độ xác thực của AP đã thay đổi.
7	ARDUINO_EVENT_WIFI_STA_GOT_IP	Trạm ESP32 nhận địa chỉ IPv4.
8	ARDUINO_EVENT_WIFI_STA_LOST_IP	Trạm ESP32 bị mất địa chỉ IP.
9	ARDUINO_EVENT_WPS_ER_SUCCESS	WPS ở chế độ đăng ký thành công.
10	ARDUINO_EVENT_WPS_ER_FAILED	WPS lỗi ở chế độ đăng ký.
11	ARDUINO_EVENT_WPS_ER_TIMEOUT	WPS hết thời gian chờ ở chế độ đăng ký.
12	ARDUINO_EVENT_WPS_ER_PIN	WPS nhận mã PIN ở chế độ đăng ký.
13	ARDUINO_EVENT_WIFI_AP_START	Khởi động soft-AP.
14	ARDUINO_EVENT_WIFI_AP_STOP	Dừng soft-AP.
15	ARDUINO_EVENT_WIFI_AP_STA_CONNECTED	Có thiết bị trạm kết nối vào soft-AP.
16	ARDUINO_EVENT_WIFI_AP_STA_DISCONNECTED	Một thiết bị trạm ngắt khỏi soft-AP.
17	ARDUINO_EVENT_WIFI_AP_STA_IP_ASSIGNED	Soft-AP gán IP cho trạm kết nối.
18	ARDUINO_EVENT_WIFI_AP_PROBREQ_RECV	Nhận gói probe request ở giao diện soft-AP.
19	ARDUINO_EVENT_WIFI_AP_GOT_IP6	Soft-AP nhận địa chỉ IPv6 ưu tiên.
19	ARDUINO_EVENT_WIFI_STA_GOT_IP6	Trạm (STA) nhận địa chỉ IPv6 ưu tiên.
19	ARDUINO_EVENT_ETH_GOT_IP6	Ethernet nhận địa chỉ IPv6 ưu tiên.
20	ARDUINO_EVENT_ETH_START	Khởi động Ethernet ESP32.
21	ARDUINO_EVENT_ETH_STOP	Dừng Ethernet ESP32.
22	ARDUINO_EVENT_ETH_CONNECTED	Ethernet đã kết nối vật lý.
23	ARDUINO_EVENT_ETH_DISCONNECTED	Ethernet bị ngắt kết nối vật lý.
24	ARDUINO_EVENT_ETH_GOT_IP	Ethernet nhận địa chỉ IPv4.
25	ARDUINO_EVENT_MAX	Giá trị giới hạn tối đa cho enum sự kiện.

2.4.2 Webservice

2.4.3 Triển khai và Code

2.5 Task 5: Triển Khai TinyML & Đánh Giá Độ Chính Xác

2.6 Task 6: Xuất Bản Dữ Liệu Lên Máy Chủ Cloud CoreIOT

3 ĐIỂM NHẤN TRIỂN KHAI

3.1 Logic Semaphore

3.2 Thiết Kế Lại Web Server

3.3 Quy Trình TinyML

3.4 Quy Trình CoreIOT

3.5 Các Công Nghệ Sử Dụng

3.6 Sơ Đồ Kiến Trúc Hệ Thống

4 PHẦN CỨNG VÀ CÔNG CỤ SỬ DỤNG

4.1 Danh sách các thành phần phần cứng

4.2 Công cụ phát triển

4.3 Môi trường phát triển

5 ĐÁNH GIÁ VÀ KẾT QUẢ THỬ NGHIỆM

5.1 Kết Quả Kiểm Thử Các Tính Năng

5.2 Đánh Giá Độ Chính Xác TinyML

5.3 Hiệu Năng Hệ Thống

5.4 Các Khó Khăn và Giải Pháp Khắc Phục

6 THẢO LUẬN NHÓM VÀ KẾT LUẬN

6.1 Kết Quả Dự Án Đạt Được

6.2 Thảo Luận Nhóm

6.3 Hạn Chế Của Dự Án

6.4 Hướng Phát Triển Trong Tương Lai

7 THÔNG TIN NHÓM

7.1 Tên Và Vai Trò Các Thành Viên Nhóm

7.2 Liên Kết Repository GitHub

7.3 Thông Tin Liên Hệ

TÀI LIỆU THAM KHẢO